

Anonymisation (classSecurity) - Hashing

Last updated by | Morgan Mick | Jul 24, 2019 at 9:33 AM GMT+1

Contents

- [Introduction](#)
- [Requirements](#)
- [Solution](#)
 - [Considerations](#)
 - [Functions](#)
- [Testing](#)
- [Limitations](#)

Introduction

A set of anonymisation functions are required to allow for having potentially sensitive data available on the Open Lake. The initial concept solution was to use the Faker library to generate fake data for attributes such as name, address etc. however after performance testing, in turnout the library makes the processing too slow to be a viable solution. Data Science use-cases which rely on correct data formatting and sentiment (i.e. keep the data meaningful as opposed to hash-masking and with the correct casing and punctuation) will be implemented on the Restricted Lake

Notes:

- Data could require anonymisation even on the Secure Lake, but only for particular requirement e.g. right to be forgotten, someone leaving the company etc. so the application of the anonymisation is not limited only to the Open Lake

Requirements

1. The anonymisation functions should take advantage of Spark to parallelise the computation so that they can be ran in short time on large amount of data

2. The anonymisation needs to be deterministic - the same input should always give the same output
3. Instead of anonymising based on the entity the attribute represents (i.e. name, address etc.), the anonymisation is to be performed based on the data type of the attribute - this will be more efficient and easy to implement as the Data Science requirements are not a factor for the Open Lake

Solution

When the data is copied from the Secure to the Open Lake, we need to anonymise it. The currently agreed approach is to use the field itself that is to be anonymised as an input for the anonymisation function and not a seed (such as the unique hash of the record). This approach has been selected since we will be just overwriting the data in the Open Lake when refresh/update is needed so there is no need to use a seed to ensure consistency of the records - when data changes in the Secure Lake we just hash whatever is the unanonymised record from the Secure Lake and update(overwrite) in the Open Lake.

- **UDFs** - In order to take advantage of Spark, custom functions will be written in Python and registered as UDFs
- Custom Logic will be developed in Python to create deterministic hashing
Important: Part of the hashing logic is using the built-in python hash() function - the function has a security feature which is enabled by default and results in having deterministic hashing **only** for the duration of the session - the same input will produce different output in different sessions. In order to disable the behaviour the following ENV variable has to be set on all clusters (it is now part of the ARM templates)
`PYTHONHASHSEED=0`

Considerations

- A complication that has to be handled is matching Python types with Spark.sql types (which correspond to the types on JVM) - python is dynamically typed language with a limited number of primitive types whereas on JVM there are multiple integer and decimal types with different precision so we must ensure we wouldn't overflow the types in the process of anonymisation
- The output of the anonymisation functions (which are based on hashing) must be matching the destination type in the Delta table - i.e. integer input needs to produce integer output (deterministic)

The following table provides the mapping between Spark.sql types and Python native types as well as shows which of the developed functions will cover each case and the corresponding UDF. In addition it shows what "type" is returned from `df.dtypes` which shows how python interprets the

are not registered with the required return type, the results will not be as expected and the type defaults to String representation (all UDFs return a string).

Developed functions:

```
hashString(stringIn, seed=None) -> returns string
hashNumeric(numberIn, seed=None) -> returns integer
hashDecimal(decimalIn, seed=None) -> returns float
hashDate(dateInput, pastOrFuture="past") -> returns Date
hashTimestamp(dateTimeInput, pastOrFuture = "past") -> returns Datetime
hashBoolean(booleanIn=None) -> always returns true
```

Notes:

- All functions use internally hashing to produce deterministic results, however the final output is of the required type for the function
- Some functions accept an optional "seed" which needs to be the unique id of the record. When a seed is provided, it is going to be used instead of the value of the attribute to be hashed. Currently, the seed is not to be used as explained previously, however the functionality is there if required at some point
- the Date/TimeStamp functions can be configured to return a data in the past or future
- There is a long list of considerations that need to be taken into account to avoid overflowing types, not matching datetime formatting and so on... A full test suite is developed to catch these

Testing

A suite of tests (34) is developed to test the functions. The test suite is located under:

AWBI -> library -> unitTests -> testHashing

Note:

- The tests output should always be the same as set in the unit tests - if they start failing at some point, please refer to the section above explaining why Python hashing may not produce deterministic results

- ## Limitations

The logic in the anonymisation functions involves taking parts of the hash produced by the hashing function, in order to generate the required output. Because of this, there is higher chance of encountering collision (i.e. 2 different inputs may generate the same "anonymised" output). This is not considered an issue at this point